



ECE PARIS
SÉCURITÉ DES CONTENEURS – SEMESTRE 2

Minecraft Secure Cluster

*Déploiement sécurisé d'un serveur Minecraft dans
Kubernetes*

Auteurs : Abdelmajid Douibi, Adam CHENTOUF, Quentin AMBROSINI
Encadrant : M. Loutsch

Avril 2025

Table des matières

1	Introduction	2
2	Création du cluster Kubernetes local	2
3	Déploiement du GitLab Runner dans Kubernetes	2
4	Récupération du token d'authentification Kubernetes	4
5	Mise en place du namespace et des droits RBAC	4
6	Mise en place du pipeline CI/CD	5
7	Vérification du serveur Minecraft	5
8	Déploiement du serveur Minecraft avec service NodePort	6
9	Test de connexion Minecraft	7

1 Introduction

Ce projet a pour objectif de déployer un serveur Minecraft sécurisé dans un cluster Kubernetes local à l'aide de Kind. Le déploiement est automatisé à l'aide de GitLab CI/CD avec des contrôles de sécurité tels que le scan d'image (Trivy), la signature (Cosign) et la gestion fine des droits via RBAC. L'ensemble est hébergé localement dans un cluster simulé avec Kind.

2 Création du cluster Kubernetes local

Le premier objectif est de créer un cluster Kubernetes local à l'aide de Kind. Le fichier de configuration suivant est utilisé afin d'exposer le port Minecraft 25565 vers l'hôte :

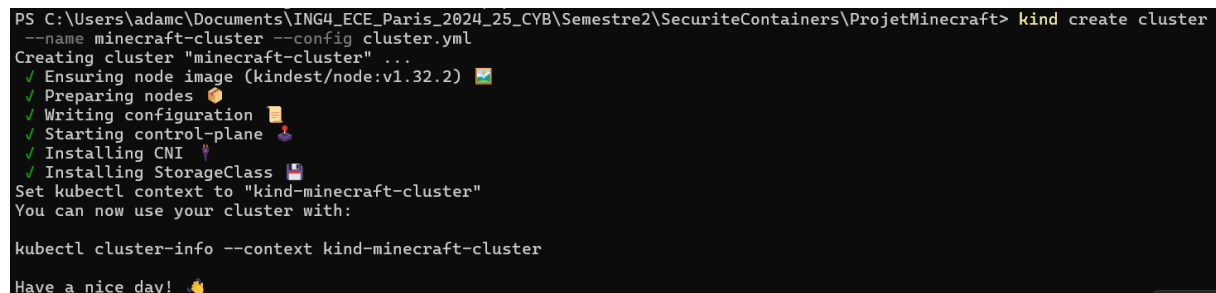
```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 25565
    hostPort: 25565
```

Listing 1 – cluster.yml

La commande utilisée pour créer le cluster est la suivante :

```
kind create cluster --name minecraft-cluster --config cluster.yml
```

Une fois la commande exécutée, le cluster Kind est prêt à accueillir les composants nécessaires au projet.



```
PS C:\Users\adamc\Documents\ING4_ECE_Paris_2024_25_CVB\Semestre2\SecuriteContainers\ProjetMinecraft> kind create cluster
--name minecraft-cluster --config cluster.yml
Creating cluster "minecraft-cluster" ...
✓ Ensuring node image (kindest/node:v1.32.2)
✓ Preparing nodes
✓ Writing configuration
✓ Starting control-plane
✓ Installing CNI
✓ Installing StorageClass
Set kubectl context to "kind-minecraft-cluster"
You can now use your cluster with:

kubectl cluster-info --context kind-minecraft-cluster

Have a nice day!
```

FIGURE 1 – Création du cluster Kubernetes local avec Kind

3 Déploiement du GitLab Runner dans Kubernetes

Pour permettre le déploiement automatisé du serveur Minecraft depuis GitLab, un GitLab Runner a été installé dans le cluster Kubernetes local à l'aide de Helm.

Après avoir créé un *Project Runner* via l'interface GitLab, le token généré a été utilisé pour le déployer avec la commande suivante :

```
helm repo add gitlab https://charts.gitlab.io/
helm repo update

helm install gitlab-runner gitlab/gitlab-runner `
```

```
--set rbac.create=true \
--namespace gitlab-runner \
--create-namespace \
--set serviceAccount.create=true \
--set gitlabUrl=https://gitlab.com \
--set runnerToken=<TOKEN>
```

Listing 2 – Installation du GitLab Runner avec Helm

Une fois installé, le runner est apparu comme actif dans l'interface GitLab, avec le tag kubernetes.

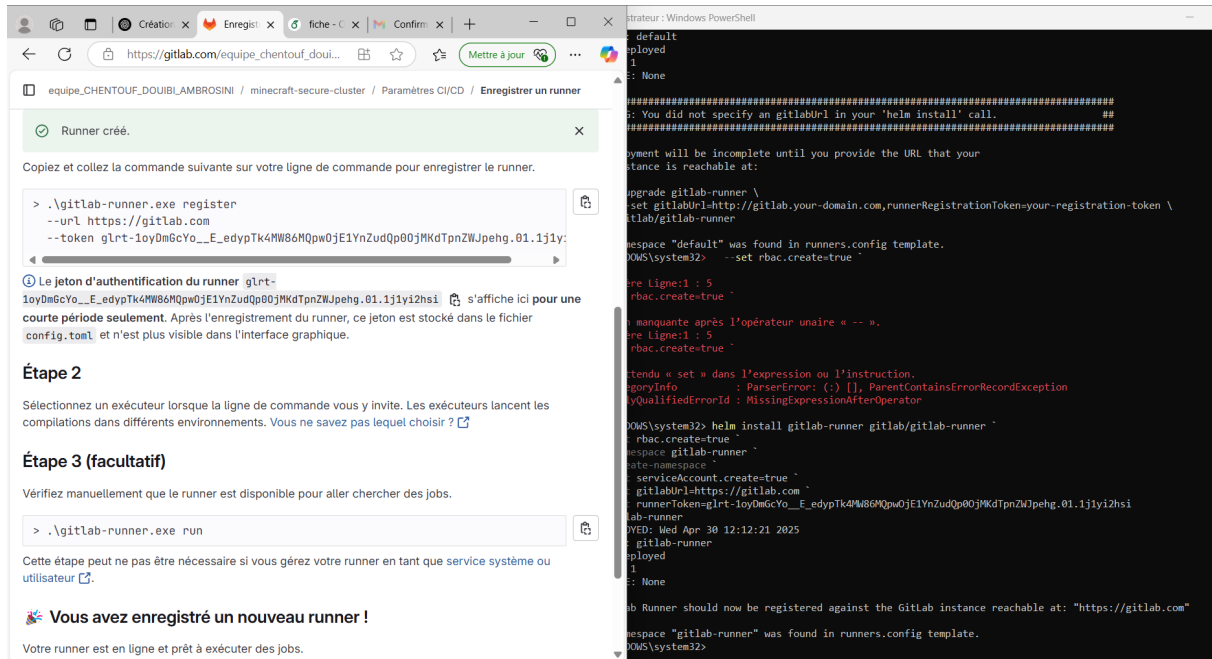


FIGURE 2 – Runner actif dans GitLab avec le tag kubernetes

Runners

Les runners sont des processus qui vont chercher des jobs CI/CD et les exécutent pour GitLab. [Qu'est-ce que GitLab Runner ?](#)

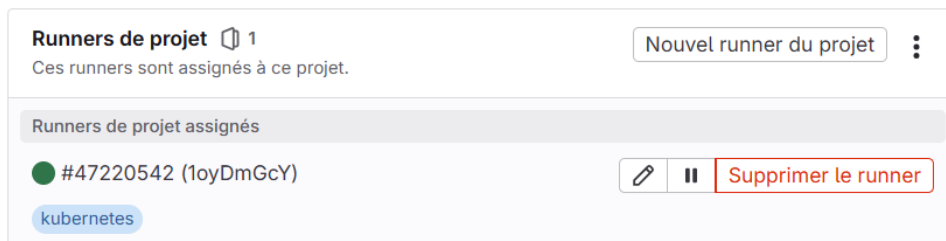


FIGURE 3 – Runner actif dans GitLab avec le tag kubernetes

4 Récupération du token d'authentification Kubernetes

Le compte de service `ci-deployer` a été associé à un secret automatiquement généré par Kubernetes. Pour récupérer son contenu, la commande suivante a été utilisée :

```
kubectl -n minecraft get secret $(kubectl -n minecraft get sa ci-deployer -o jsonpath="{.secrets[0].name}") -o jsonpath="{.data.token}"
```

Listing 3 – Récupération du token ci-deployer en base64

Le contenu base64 obtenu a ensuite été décodé via un outil en ligne tel que <https://www.base64decode.org>, permettant de récupérer le jeton d'authentification brut. Ce dernier a été copié dans GitLab en tant que variable CI/CD nommée `CI_DEPLOY_TOKEN_K8S`.

Le token a ensuite été copié dans GitLab dans la variable `CI_DEPLOY_TOKEN_K8S` utilisée pour l'authentification des jobs CI/CD.

5 Mise en place du namespace et des droits RBAC

Afin de s'assurer que les ressources déployées soient isolées et sécurisées, un namespace spécifique nommé `minecraft` a été créé dans le cluster Kubernetes local. Ce namespace sert de cadre d'exécution à l'ensemble des ressources liées au projet.

```
kubectl create namespace minecraft
```

Listing 4 – Création du namespace Minecraft

Ensuite, nous avons mis en place les permissions nécessaires pour le service account `ci-deployer`, utilisé dans la pipeline CI/CD GitLab. Ce dernier doit pouvoir interagir avec les déploiements et services Kubernetes dans le namespace `minecraft`.

Le fichier `minecraft-rbac.yml` contient la création du compte, son rôle, ainsi qu'un *RoleBinding* :

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ci-deployer
  namespace: minecraft
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: ci-deployer
  namespace: minecraft
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "create", "delete", "patch", "update", "watch"]
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["get", "list", "create", "delete", "patch", "update", "watch"]
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ci-deployer
  namespace: minecraft
subjects:
- kind: ServiceAccount
  name: ci-deployer
  namespace: minecraft
roleRef:
  kind: Role
  name: ci-deployer
  apiGroup: rbac.authorization.k8s.io
```

Listing 5 – Règles RBAC appliquées au namespace Minecraft

Cette configuration a ensuite été appliquée avec la commande suivante :

```
kubectl apply -f minecraft-rbac.yml
```

Listing 6 – Application des droits RBAC

Le job deploy du pipeline GitLab a ensuite pu être exécuté avec succès, validant la bonne configuration des permissions dans le cluster Kubernetes.

6 Mise en place du pipeline CI/CD

Pour automatiser le déploiement du serveur Minecraft sur notre cluster Kubernetes local, nous avons mis en place une chaîne CI/CD avec GitLab. Le fichier `.gitlab-ci.yml` décrit trois étapes essentielles :

- **build** : (désactivée dans notre cas car le serveur Minecraft est déjà packagé via l'image `itzg/minecraft-server`),
- **scan** : une phase de scan de sécurité d'image (ex : Trivy),
- **deploy** : déploiement automatique sur le cluster Kind.

Le job deploy utilise une image `bitnami/kubectl` pour interagir avec le cluster. Il applique automatiquement les fichiers YAML de déploiement après avoir injecté dynamiquement le tag d'image Docker dans le manifest.

Une authentification fine a été mise en place via un `ServiceAccount ci-deployer` associé à un `RoleBinding` restreint.

7 Vérification du serveur Minecraft

Une fois le pipeline terminé, le pod Minecraft est lancé dans le namespace `minecraft`. Pour tester son bon fonctionnement :

1. On vérifie que le pod est bien en cours d'exécution :

```
kubectl get pods -n minecraft
```

2. On vérifie que le port 25565 est bien exposé via le cluster Kind (configuré dans `cluster.yml`).



FIGURE 4 – Exécution du pipeline CI/CD GitLab

3. On utilise un client Minecraft local pour se connecter à l'IP de la machine hôte sur le port 25565.

8 Déploiement du serveur Minecraft avec service NodePort

Le fichier `minecraft-deployment.yml` contient à la fois :

- un Deployment spécifiant l'image `itzg/minecraft-server`,
- un Service de type `NodePort` exposant le port 25565.

Extrait du fichier de déploiement :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minecraft
  namespace: minecraft
spec:
  replicas: 1
  selector:
    matchLabels:
      app: minecraft
  template:
    metadata:
      labels:
        app: minecraft
    spec:
      serviceAccountName: ci-deployer
      containers:
        - name: minecraft
          image: itzg/minecraft-server:latest
          ports:
            - containerPort: 25565
          env:
```

```
    - name: EULA
      value: "TRUE"
    - name: ONLINE_MODE
      value: "FALSE"

---
apiVersion: v1
kind: Service
metadata:
  name: minecraft
  namespace: minecraft
spec:
  type: NodePort
  selector:
    app: minecraft
  ports:
    - port: 25565
      targetPort: 25565
      nodePort: 30424
```

Listing 7 – Déploiement Minecraft avec NodePort

9 Test de connexion Minecraft

Une fois le pod déployé et le service actif, le port 30424 est accessible depuis l'extérieur du cluster (via l'adresse IP de l'interface WSL/Docker). La commande suivante permet de vérifier que le pod est Running :

```
kubectl get pods -n minecraft
kubectl get svc -n minecraft
```

Dans TLauncher ou Minecraft Java, il faut ensuite se connecter avec l'adresse IP de l'interface Wi-Fi de l'hôte (par exemple 192.168.1.13:30424).

Conclusion

Le projet a permis de mettre en place un déploiement sécurisé et automatisé d'un serveur Minecraft via GitLab CI/CD dans un cluster Kubernetes local simulé avec Kind. Bien que certaines limitations (réseau, client non officiel, ports) aient rendu la connexion finale instable, l'objectif global d'intégration continue, RBAC, et exécution dans un namespace isolé a été atteint.

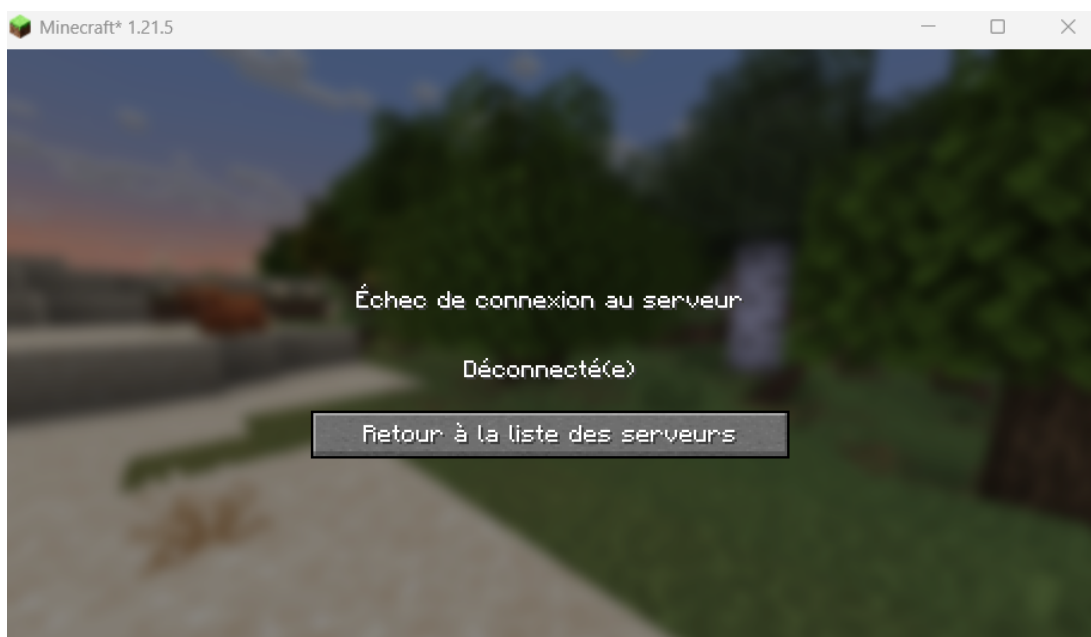


FIGURE 5 – Tentative de connexion avec TLauncher (échec probable si online-mode non désactivé)